

目录

1. 系统架构选择及说明.....	5
1.1 系统特征与架构驱动力	5
1.2 候选架构方案对比	5
1.2.1 方案 A：客户端 / 服务器（C/S）三层架构 + MVC	5
1.2.2 方案 B：微服务架构	5
1.2.3 方案 C：单体单层架构（无明显分层）	5
1.2.4 综合对比	6
1.3 选定架构：C/S 三层架构 + MVC（方案 A）	6
1.3.1 选型理由	6
1.3.2 架构分层与职责	6
1.3.3 逻辑视图	7
1.3.4 部署视图	8
1.4 架构对后续设计的约束	8
2. 静态结构设计.....	9
2.1 建模约束	9
2.2 全局类图（概览）	9
2.3 子系统类图	10
2.3.1 用户域类图（UserSubsystem）	10
2.3.2 调度域类图（SchedulingSubsystem）	11
2.3.3 充电桩域类图（PileSubsystem）	12

2.3.4 计费/报表域类图（BillingSubsystem）	13
2.4 类属性与方法说明（全局概览图配套）	13
2.5 用例 → 参与类映射表	16
2.6 关键设计模式说明	17
3. 动态结构设计	17
3.1 建模约定	17
3.2 用户域用例顺序图（UC01~UC09）	18
3.2.1 UC01 注册	18
3.2.2 UC02 登录	19
3.2.3 UC03 提交充电请求	20
3.2.4 UC04 修改充电请求	21
3.2.5 UC05 取消充电	22
3.2.6 UC06 查看本车排队号	23
3.2.7 UC07 查看本充电模式前车数	23
3.2.8 UC08 结束充电	24
3.2.9 UC09 查看充电详单	25
3.3 管理员域用例顺序图（UC10~UC14）	26
3.3.1 UC10 启动/关闭充电桩	26
3.3.2 UC11 查看所有充电桩状态	27
3.3.3 UC12 查看排队车辆信息	28
3.3.4 UC13 查看运营报表	29
3.3.5 UC14 切换调度策略	30

3.4 充电桩故障调度顺序图	31
3.4.1 故障调度 — 优先级调度	31
3.4.2 故障调度 — 时间顺序调度	32
3.4.3 故障恢复 — 重调度	33
3.5 扩展调度策略顺序图	34
3.5.1 单次调度 — 总充电时长最短（MinSinglePolicy）	34
3.5.2 批量调度 — 总充电时长最短（MinBatchPolicy）	36
4. 工作量统计.....	37
4.1 工作量百分比汇总	39

1. 系统架构选择及说明

1.1 系统特征与架构驱动力

按照需求文档，本系统具备以下关键特征：

维度	取值	对架构的影响
用户客户端类型	普通用户端 + 管理员端	客户端需要分角色界面，但共享服务端能力
业务并发规模	单充电站，并发约 $(3 \text{ 快充桩} + 2 \text{ 慢充桩}) \times M \text{ 排队位} + N \text{ 等候位}$	中小规模，无须分布式集群
实时性	排队号/前车数/故障事件需"准实时"推送	需支持服务端推送（WebSocket 或长轮询）
调度逻辑复杂度	含 5 种调度场景：基线、故障优先级、故障时间顺序、单次最短、批量最短	调度策略须支持运行时切换，倾向 Strategy 模式
数据持久化	详单、报表必须落库	需要稳定的关系存储
升级与变更	课程作业；评分点偏架构合理性，不强调可扩展性极致	架构应"够用"，避免过度设计

1.2 候选架构方案对比

1.2.1 方案 A：客户端 / 服务器 (C/S) 三层架构 + MVC

把系统切成"表示层 — 业务逻辑层 — 数据访问层"三层，业务逻辑层内部按 MVC 思路把 Controller、Service、Strategy、Entity 分离；客户端只承担表示层，服务器集中承担调度、计费、详单、报表与充电桩监控。

- 优点：实现简单、部署轻量；调度策略与计费规则集中，便于一致性维护；与领域模型/用例模型天然对齐（"服务器端"在需求中即一个统一对象）。
- 缺点：所有业务集中在单进程，水平扩展受限。

1.2.2 方案 B：微服务架构

把"用户服务、请求/排队服务、调度服务、充电桩管理服务、计费/报表服务"拆为 5 个独立部署的微服务，通过消息队列或 gRPC 通信。

- 优点：可独立伸缩；调度服务可独立替换调度算法。
- 缺点：单充电站规模下纯属"用大炮打蚊子"；引入了分布式事务、服务发现、消息中间件等额外复杂度；课程内调试与部署成本显著抬高。

1.2.3 方案 C：单体单层架构（无明显分层）

服务器内不区分控制层 / 服务层 / 数据层，UI 直接连数据库或简单脚本。

- 优点：原型阶段开发最快。
- 缺点：调度逻辑、计费规则、桩监控耦合在一起；故障调度等复杂交互无法清晰建模，无法满足作业 2 对"动态结构"的清晰展示要求。

1.2.4 综合对比

维度	方案 A：C/S 三层 + MVC	方案 B：微服务	方案 C：单体单层
与需求规模匹配度	高	低	中
实现/部署复杂度	低	高	极低
业务一致性维护	易	难（跨服务事务）	易
调度策略可切换性	高，Strategy 模式天然适配	高	低
与模型对齐度	高	中	中
评分契合度(结构清晰、动态可建模)	高	中	低

1.3 选定架构：C/S 三层架构 + MVC（方案 A）

1.3.1 选型理由

- 1. 规模匹配：单充电站、单服务器即可承载 5 个桩 × M + N 辆车的并发量级，无需引入分布式开销。
- 2. 与需求模型对齐：需求文档明确将系统切分为"用户客户端 + 管理员客户端 + 服务器端"三方；C/S 模型与之天然对应，便于在顺序图与类图中把对象边界画清。
- 3. 支持调度策略动态切换：业务逻辑层内的 Scheduler 通过 Strategy 模式承载 BaselinePolicy / MinSinglePolicy / MinBatchPolicy 三种调度策略，配合 UC14 切换策略用例。
- 4. 故障调度可建模：PileManager、Scheduler、FaultMonitor 三者间的协作可在动态结构里以清晰的顺序图表示，符合任务对"故障情况下调度过程"的展示要求。
- 5. 限制过度设计：方案 A 既满足又不引入冗余复杂度。

1.3.2 架构分层与职责

层	关键组件	职责
表示层	CustomerUI, AdminUI	渲染界面、收集用户输入、展示推送数据；不含业务规则
业务逻辑层	UserController, ChargingController, Scheduler, PileManager, FaultMonitor, BillingEngine, ReportGenerator	用例控制流、调度策略、计费规则、报表生成；通过领域对象操作数据
数据访问层	UserRepo, RequestRepo, PileRepo, BillRepo	屏蔽数据库细节；对上层暴露领域对象

1.3.3 逻辑视图

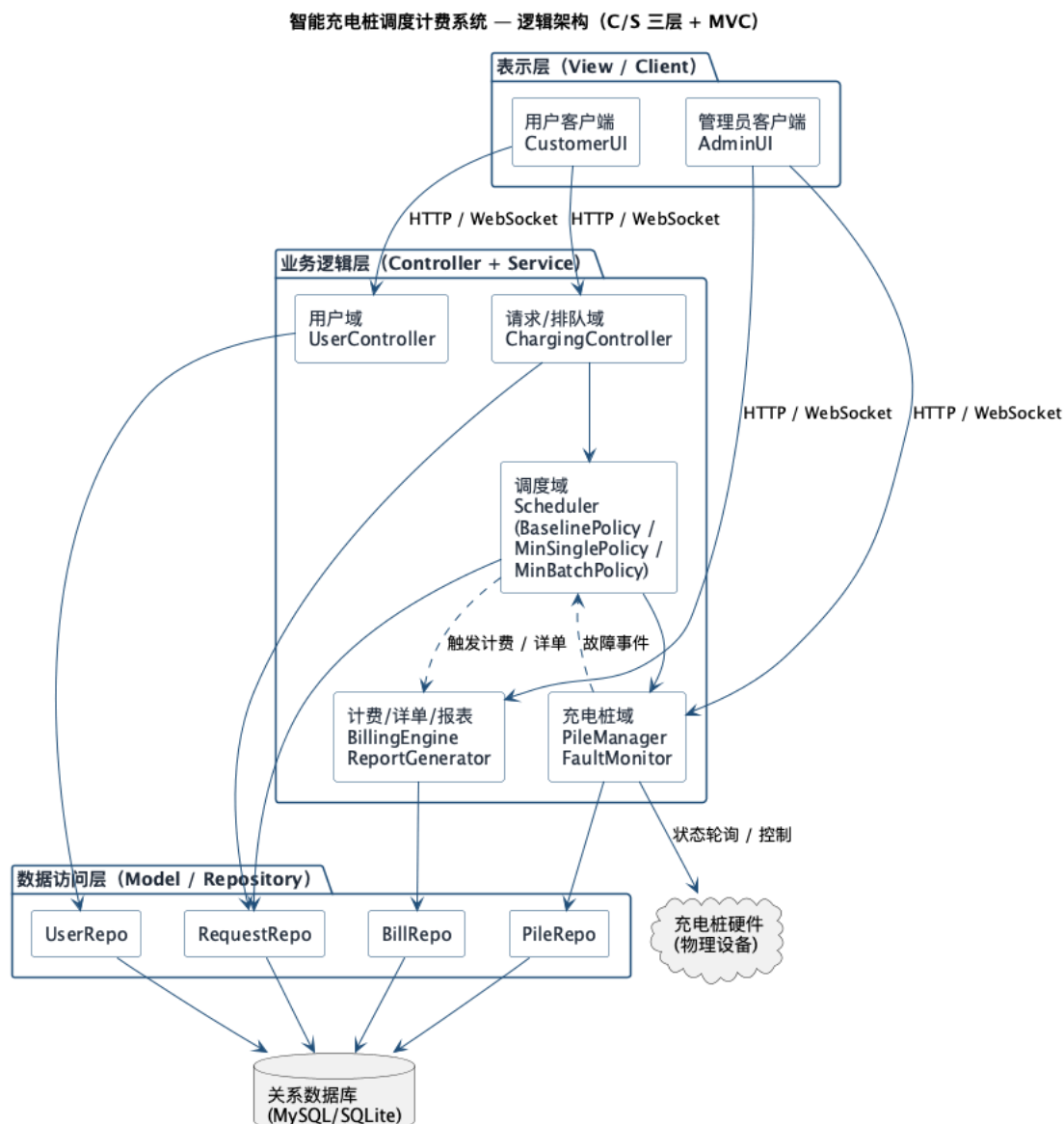


图 1 智能充电桩调度计费系统 — 逻辑架构

逻辑视图说明：

- 表示层（两类客户端）通过 HTTPS + WebSocket 与业务逻辑层通信；普通用户与管理层路由分离但底层服务统一。
- Scheduler 作为调度策略的执行者，依赖 PileManager 提供桩状态、依赖 RequestRepo 提供请求队列；它与 BillingEngine 之间用依赖关系连接，故障/正常结束时触发计费。
- PileManager 与 FaultMonitor 持续轮询桩硬件并向 Scheduler 推送故障事件，对应需求中"充电桩故障"场景。
- 数据访问层只服务于上层 Service / Controller，向下封装 SQL；这一层在动态顺序图中通常省略，但在静态类图里显式存在。

1.3.4 部署视图

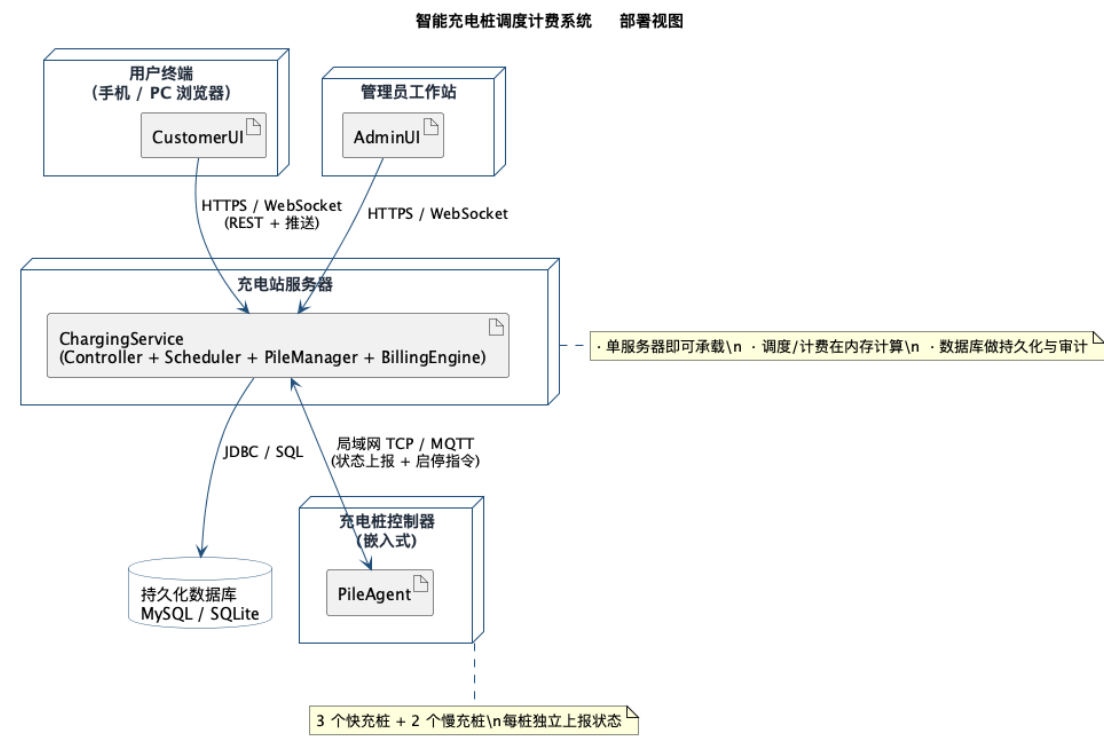


图 2 智能充电桩调度计费系统 — 部署视图

部署视图说明：

- 用户终端与管理员工作站通过浏览器/客户端访问"充电站服务器"上的统一服务进程；服务进程内嵌业务逻辑层与数据访问层，对外发布 REST/WebSocket 接口。
- 持久化数据库与服务器同机房部署即可，存储用户、请求、桩状态、详单、报表。
- 5 个充电桩各自运行一个轻量 **PileAgent**（嵌入式控制端），通过局域网 TCP 或 MQTT 与服务器交互，上报状态并接收启停/调度指令。

1.4 架构对后续设计的约束

- 静态结构中的类图按上述三层组织：Boundary（UI）、Controller/Service/Strategy、Entity/Repository；类间关系仅使用依赖、定向关联、继承三种。
- 动态结构中的顺序图采用"BCE 白盒"模型：消息序列从 :CustomerUI/:AdminUI 进入 :XController，再分派到 :Scheduler / :PileManager / :BillingEngine，最终由 :Repository 落库；该层次结构与本章架构图保持一致。

- **用户域 UserSubsystem:** User 抽象类被 Customer、Administrator 继承; CustomerUI / AdminUI 定向关联 UserController, UserController 定向关联 UserRepo。
- **调度域 SchedulingSubsystem:** DispatchPolicy 抽象类被 BaselinePolicy / MinSinglePolicy / MinBatchPolicy 继承; Scheduler 定向关联 DispatchPolicy, 并依赖 ChargingRequest、PileManager (跨子系统)。
- **充电桩域 PileSubsystem:** ChargingPile 抽象类被 FastChargingPile / TrickleChargingPile 继承; PileManager 定向关联 ChargingPile / FaultMonitor / PileRepo; FaultMonitor 依赖 Scheduler 完成故障调度。
- **计费/报表域 BillingSubsystem:** BillingEngine 定向关联 PricingPolicy / BillDetail / BillRepo; Report 依赖 BillDetail。

2.3 子系统类图

2.3.1 用户域类图 (UserSubsystem)

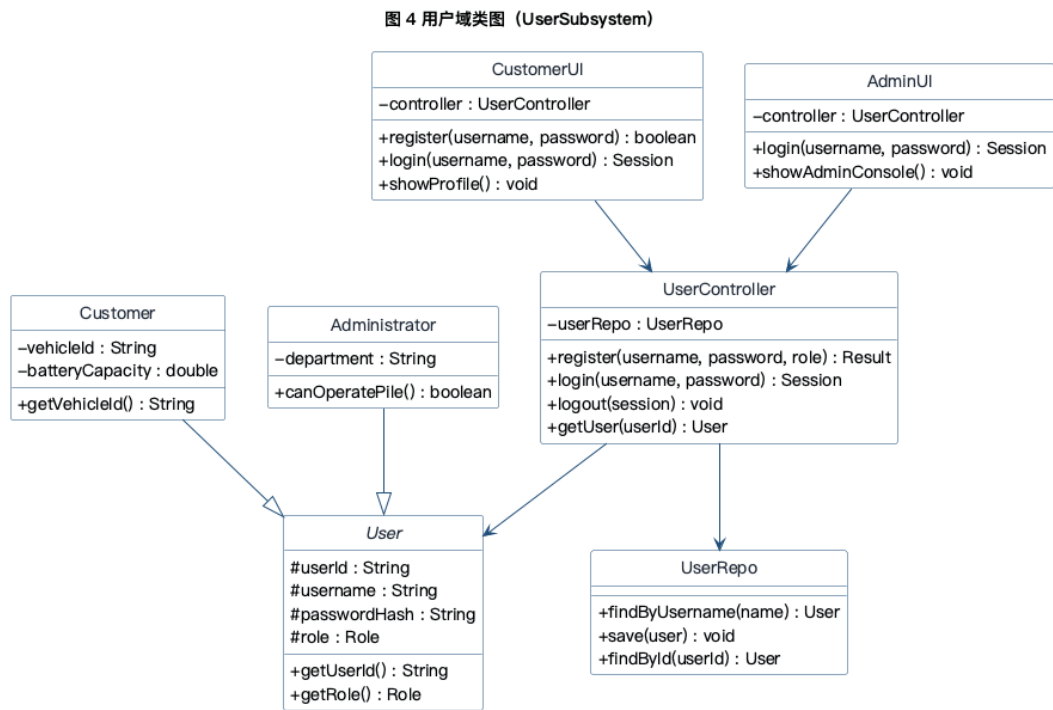


图 4 用户域类图

承担 UC01 注册、UC02 登录。User 为抽象基类，封装 `userId / username / passwordHash / role`；Customer 持有车辆相关字段 (`vehicleId / batteryCapacity`)；Administrator 持有管理职权字段 (`department / canOperatePile()`)。UserRepo 屏蔽数据库细节。

2.3.2 调度域类图 (SchedulingSubsystem)

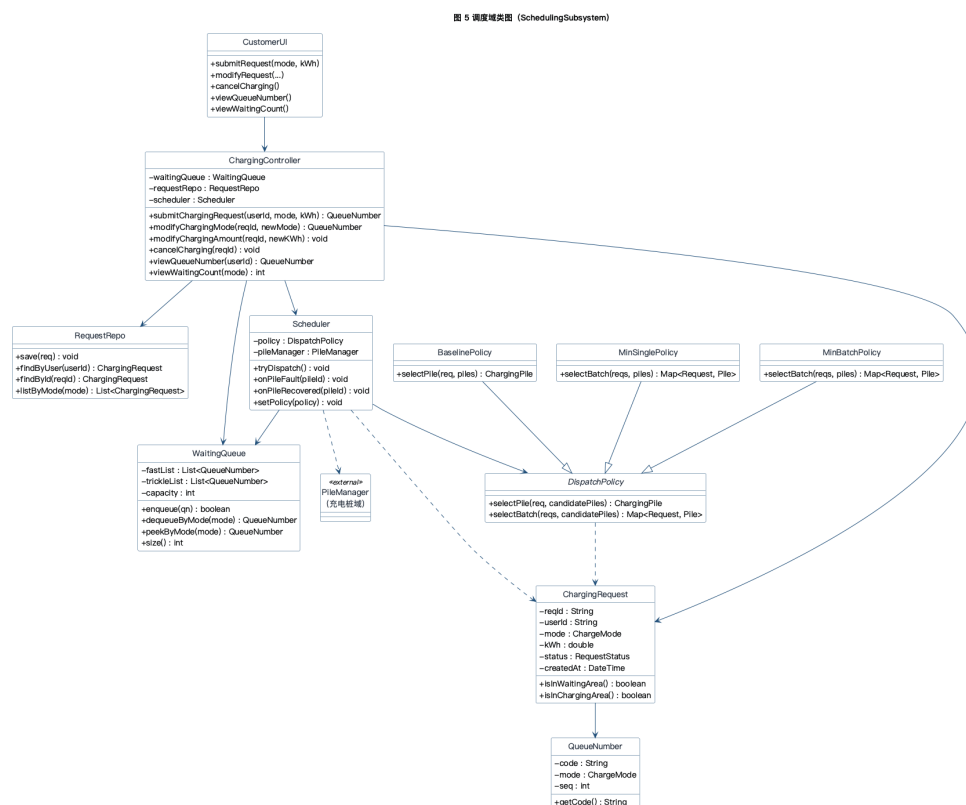


图 5 调度域类图

承担 UC03~UC07 与 UC14（设置调度策略），是动态结构的核心：

- ChargingController 接受 CustomerUI 的请求消息（提交/修改/取消/查询），落盘到 RequestRepo，并交给 Scheduler 触发调度。
- Scheduler 持有 DispatchPolicy 引用，运行时可被 UC14 切换为 BaselinePolicy / MinSinglePolicy / MinBatchPolicy；这是典型的 Strategy 模式（继承关系在图中以 --|> 表示）。
- WaitingQueue 内部维护 fastList 与 trickleList 两条 FIFO 队列，对应需求中“等候区按 F/T 类型排队”。

2.3.3 充电桩域类图 (PileSubsystem)

图 6 充电桩域类图 (PileSubsystem)

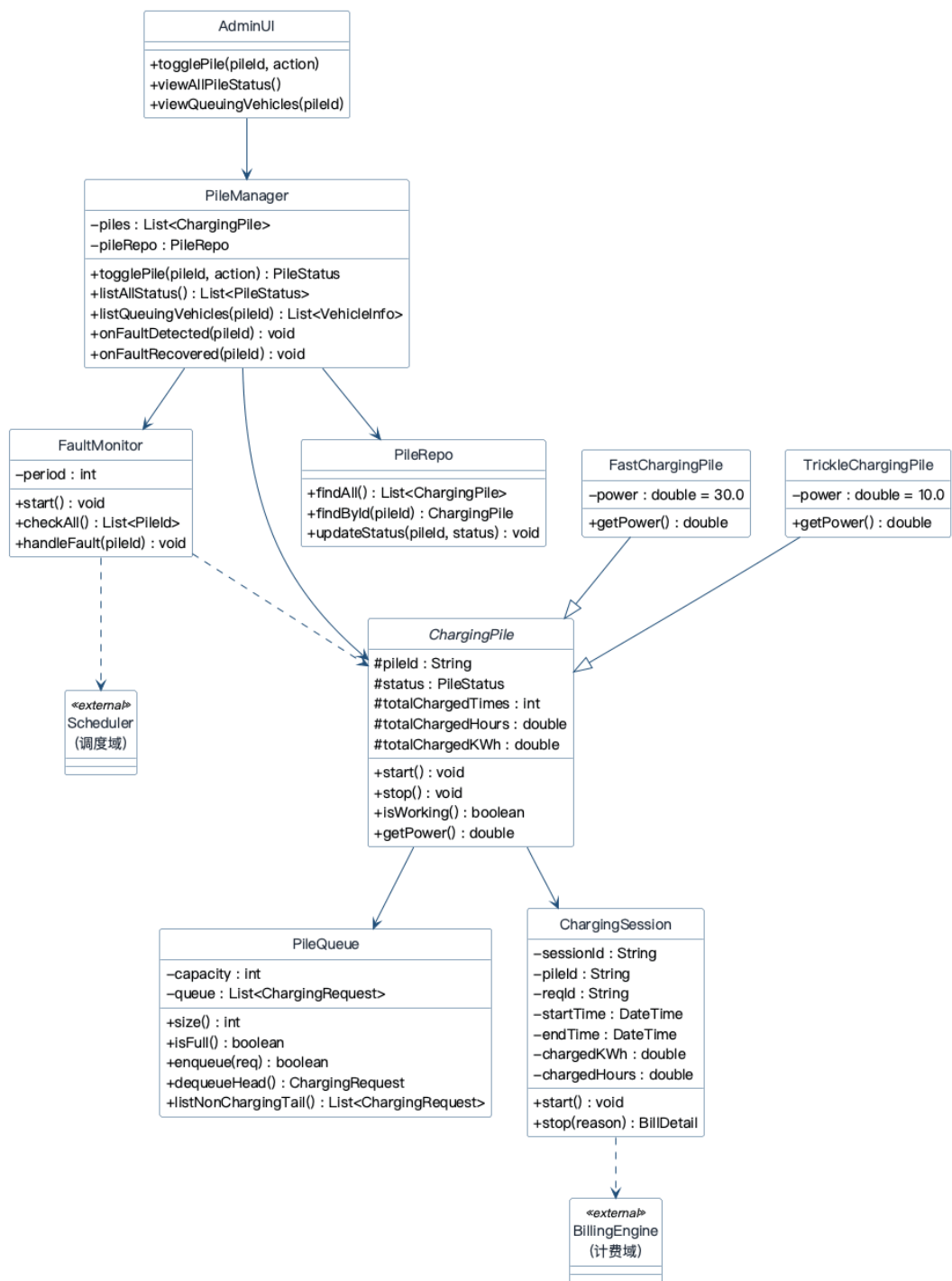


图 6 充电桩域类图

承担 UC08 结束充电、UC10 启停充电桩、UC11 查看所有桩状态、UC12 查看排队车辆，以及组长加做的"故障调度"协作：

- ChargingPile 为抽象类；FastChargingPile.power = 30.0，TrickleChargingPile.power = 10.0（与需求一致）。
- FaultMonitor 周期巡检桩状态，发现故障时调用 Scheduler.onPileFault(pileId)，

恢复后调用 onPileRecovered，串联起故障 → 调度的事件链。

- ChargingSession 负责单次充电过程的开始/结束、累计度数与时长；结束时依赖 BillingEngine 生成详单。

2.3.4 计费/报表域类图 (BillingSubsystem)

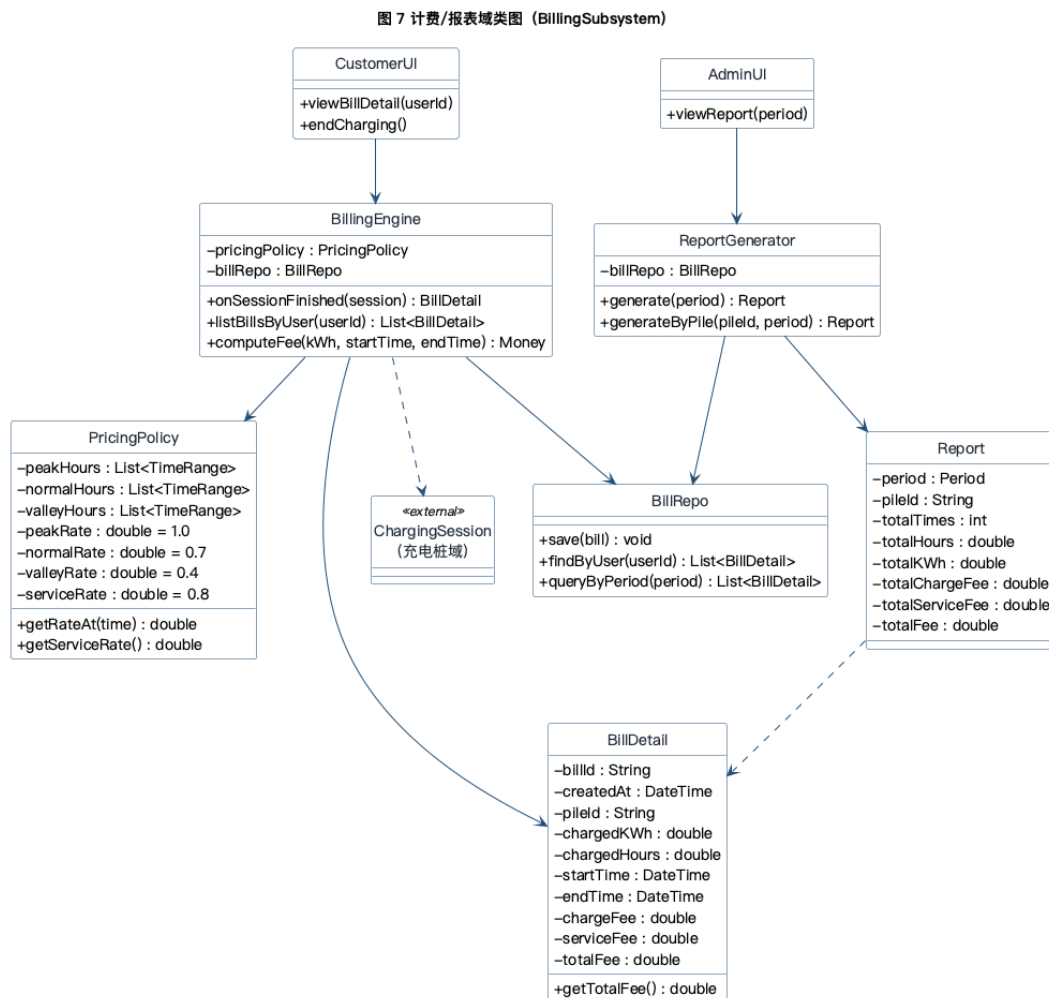


图 7 计费/报表域类图

承担 UC09 查看充电详单、UC13 查看运营报表：

- BillingEngine 在收到 ChargingSession 结束事件时，按 PricingPolicy 当前时段费率计算 `chargeFee + serviceFee`，落库为 BillDetail。
- PricingPolicy 内置峰/平/谷三段费率（1.0 / 0.7 / 0.4 元/度）与服务费 0.8 元/度，符合需求计费规则。
- ReportGenerator 按日/周/月聚合 BillRepo 中的 BillDetail，生成 Report。

2.4 类属性与方法说明（全局概览图配套）

全局概览采用 "hide members" 的名称视图，本节以表格形式补充其类属性与方法。

表 1 用户域类属性与方法

类	属性	主要方法
CustomerUI	controller:UserController	register(), login(), showProfile()

AdminUI	controller:UserController	login(), showAdminConsole()
UserController	userRepo:UserRepo	register(name,pwd,role), login(name,pwd), logout(session), getUser(userId)
User *(抽象)*	userId, username, passwordHash, role	getUserId(), getRole()
Customer	vehicleId, batteryCapacity	getVehicleId()
Administrator	department	canOperatePile()
UserRepo	—	findByUsername(), save(), findById()

表 2 调度域类属性与方法

类	属性	主要方法
ChargingController	waitingQueue, requestRepo, scheduler	submitChargingRequest(), modifyChargingMode(), modifyChargingAmount(), cancelCharging(), viewQueueNumber(), viewWaitingCount()
Scheduler	policy:DispatchPolicy, pileManager	tryDispatch(), onPileFault(pileId), onPileRecovered(pileId), setPolicy(policy)
DispatchPolicy *(抽象)*	—	selectPile(req, candidates), selectBatch(reqs, candidates)
BaselinePolicy	—	selectPile(): 在匹配模式的所有桩中选 min(等待时间 + 自己充电时间)
MinSinglePolicy	—	selectBatch(): 多空位同时叫号, min Σ (等待+充电)
MinBatchPolicy	—	selectBatch(): 充电区+等候区满时触发批量调度, 不区分快慢
WaitingQueue	fastList, trickleList, capacity	enqueue(), dequeueByMode(), peekByMode(), size()
ChargingRequest	reqId, userId, mode, kWh, status, createdAt	isInWaitingArea(), isInChargingArea()
QueueNumber	code, mode, seq	getCode()
RequestRepo	—	save(), findByUser(), findById(), listByMode()

表 3 充电桩域类属性与方法

类	属性	主要方法
---	----	------

PileManager	piles, pileRepo	togglePile(pileId, action), listAllStatus(), listQueuingVehicles(pileId), onFaultDetected(pileId), onFaultRecovered(pileId)
FaultMonitor	period	start(), checkAll(), handleFault(pileId)
ChargingPile *(抽象)*	pileId, status, totalChargedTimes, totalChargedHours, totalChargedKWh	start(), stop(), isWorking(), getPower()
FastChargingPile	power = 30.0	getPower()
TrickleChargingPile	power = 10.0	getPower()
PileQueue	capacity, queue	size(), isFull(), enqueue(), dequeueHead(), listNonChargingTail()
ChargingSession	sessionId, pileId, reqId, startTime, endTime, chargedKWh, chargedHours	start(), stop(reason)
PileRepo	—	findAll(), findById(), updateStatus()

表 4 计费/报表域类属性与方法

类	属性	主要方法
BillingEngine	pricingPolicy, billRepo	onSessionFinished(session), listBillsByUser(userId), computeFee(kWh, startTime, endTime)
PricingPolicy	peakHours, normalHours, valleyHours, peakRate=1.0, normalRate=0.7, valleyRate=0.4, serviceRate=0.8	getRateAt(time), getServiceRate()
BillDetail	billId, createdAt, pileId, chargedKWh, chargedHours, startTime, endTime, chargeFee, serviceFee, totalFee	getTotalFee()
ReportGenerator	billRepo	generate(period), generateByPile(pileId, period)
Report	period, pileId, totalTimes, totalHours, totalKWh, totalChargeFee,	—

	totalServiceFee, totalFee	
BillRepo	—	save(), findByUser(), queryByPeriod()

2.5 用例 → 参与类映射表

表 5 用例与参与类映射

用例	用例名称	主要参与类（按调用先后）	所属子系统图
UC01	注册	CustomerUI → UserController → User、UserRepo	用户域(图 4)
UC02	登录	CustomerUI/AdminUI → UserController → UserRepo	用户域(图 4)
UC03	提交充电请求	CustomerUI → ChargingController → WaitingQueue、ChargingRequest、RequestRepo、Scheduler	调度域(图 5)
UC04	修改充电请求	CustomerUI → ChargingController → ChargingRequest、WaitingQueue、QueueNumber	调度域(图 5)
UC05	取消充电	CustomerUI → ChargingController → WaitingQueue / PileManager.pileQueue、ChargingRequest	调度域(图 5) + 充电桩域(图 6)
UC06	查看排队号	CustomerUI → ChargingController → RequestRepo → QueueNumber	调度域(图 5)
UC07	查看前车数	CustomerUI → ChargingController → WaitingQueue、PileManager	调度域(图 5) + 充电桩域(图 6)
UC08	结束充电	CustomerUI → ChargingController → PileManager → ChargingPile → ChargingSession.stop() → BillingEngine	充电桩域(图 6) + 计费域(图 7)
UC09	查看充电详单	CustomerUI → BillingEngine → BillRepo → BillDetail	计费域(图 7)
UC10	启动/关闭充电桩	AdminUI → PileManager → ChargingPile、PileRepo	充电桩域(图 6)
UC11	查看所有充电桩状态	AdminUI → PileManager.listAllStatus()	充电桩域(图 6)
UC12	查看排队车辆信息	AdminUI → PileManager.listQueuingVehicles() → PileQueue → ChargingRequest	充电桩域(图 6)
UC13	查看运营报表	AdminUI → ReportGenerator → BillRepo → Report	计费域(图 7)
UC14	切换调度策略	AdminUI → Scheduler.setPolicy() → BaselinePolicy / MinSinglePolicy / MinBatchPolicy	调度域(图 5)
F1	故障调度	— FaultMonitor → Scheduler.onPileFault()	调度域(图 5)

	优先级	→ BaselinePolicy.selectPile() → PileManager	+ 充电桩域 (图 6)
F2	故障调度 — 时间顺序	同 F1 , 调度顺序按 QueueNumber.seq 重排	同上
F3	故障恢复重调 度	FaultMonitor → Scheduler.onPileRecovered()	同上

2.6 关键设计模式说明

模式	落地位置	作用
Strategy (策略模式)	Scheduler 持有 DispatchPolicy 抽象引用, Baseline / MinSingle / MinBatch 三种子类	支持 UC14 运行时切换调度策略, 三种策略对调用方透明
Template Method (模板方法)	ChargingPile.start()/stop() 与 ChargingSession.start()/stop(reason) 的协作	不同子类 (快/慢充电桩) 共用控制流, 仅 power 参数与计费 速率不同
Repository (仓储模式)	UserRepo / RequestRepo / PileRepo / BillRepo	业务逻辑层不直接接触数据库, 所有动态 顺序图末端落在 Repository
Observer (观察者, 事件式)	FaultMonitor → Scheduler 的故障 事件通知	把 "桩故障检测" 与 " 调度处理" 解耦, 对应 需求中的故障调度场 景

3. 动态结构设计

3.1 建模约定

维度	约定
对象命名	实例用 :ClassName; 同一类不同实例用 obj:ClassName; UML 风格统一
对象类型	Boundary (UI) / Controllor / Entity / Service / Strategy / Repository
消息命名	全部采用可编程英文 (驼峰、动词开头), 重要参数与返回值 显式标注
控制结构	用 alt、opt、loop、par 标注分支/可选/循环/并发
条件可视化	需求中的关键条件 (队列容量、桩故障、模式约束) 写在 note 中靠近其判定位置

3.2 用户域用例顺序图 (UC01~UC09)

3.2.1 UC01 注册

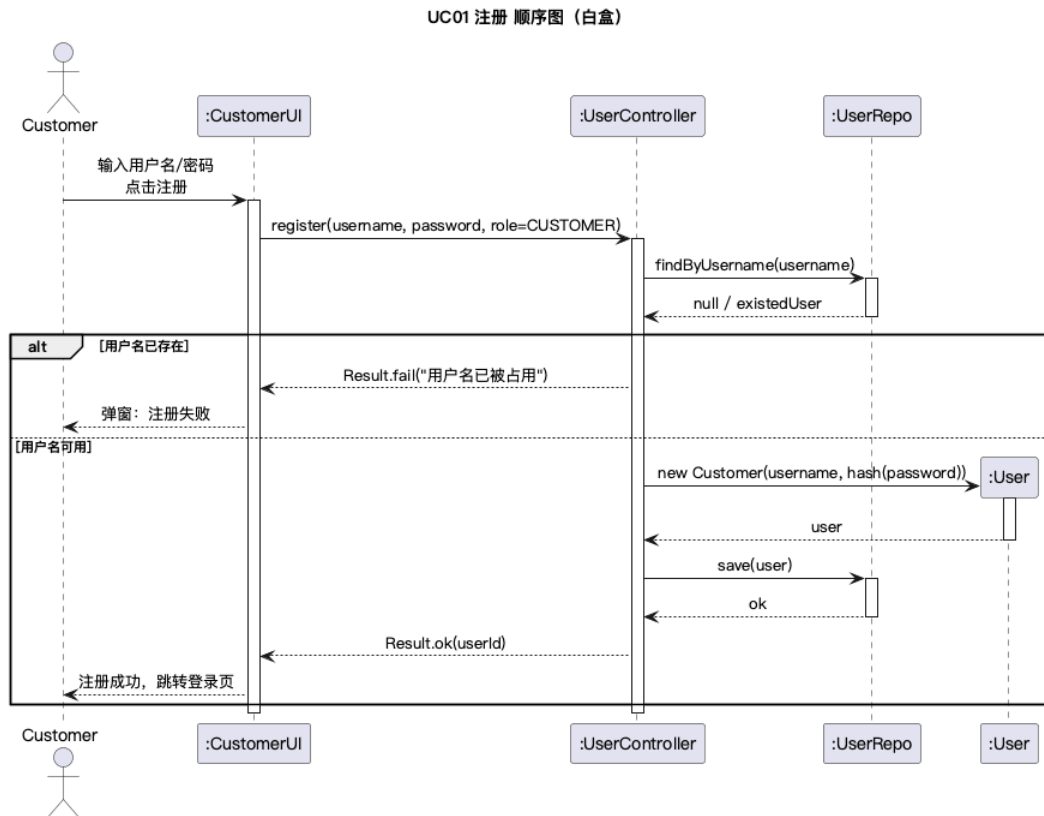


图 8 UC01 注册 顺序图

关键流程: :CustomerUI → :UserController → :UserRepo; 用 alt 区分"用户名已存在 / 可用"两个分支; 新用户对象通过 new Customer(...) 创建。

3.2.2 UC02 登录

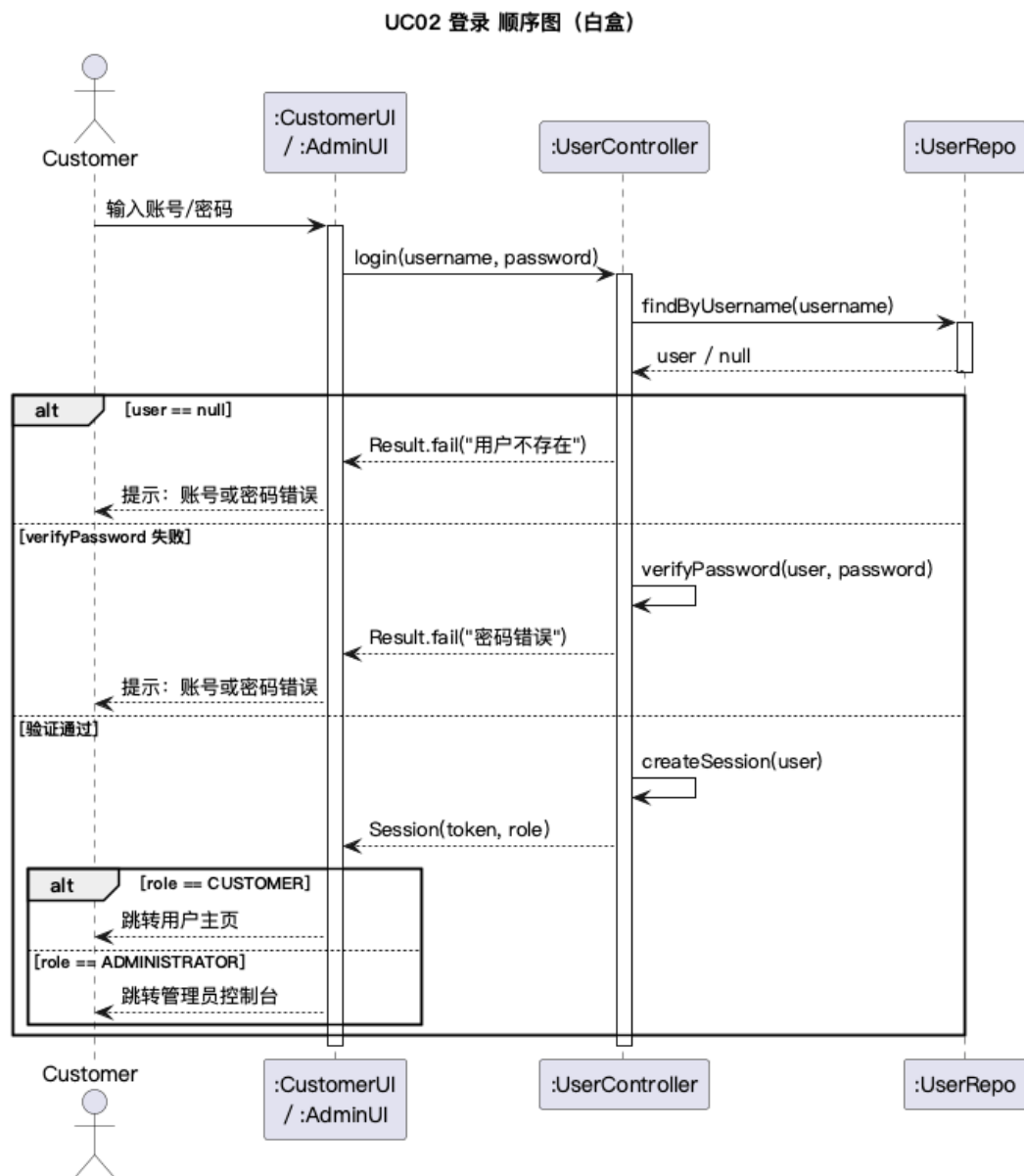


图 9 UC02 登录 顺序图

三支 alt：用户不存在 / 密码错误 / 验证通过；通过后内部 createSession() 并按 role 跳转不同 UI 子页面。

3.2.3 UC03 提交充电请求

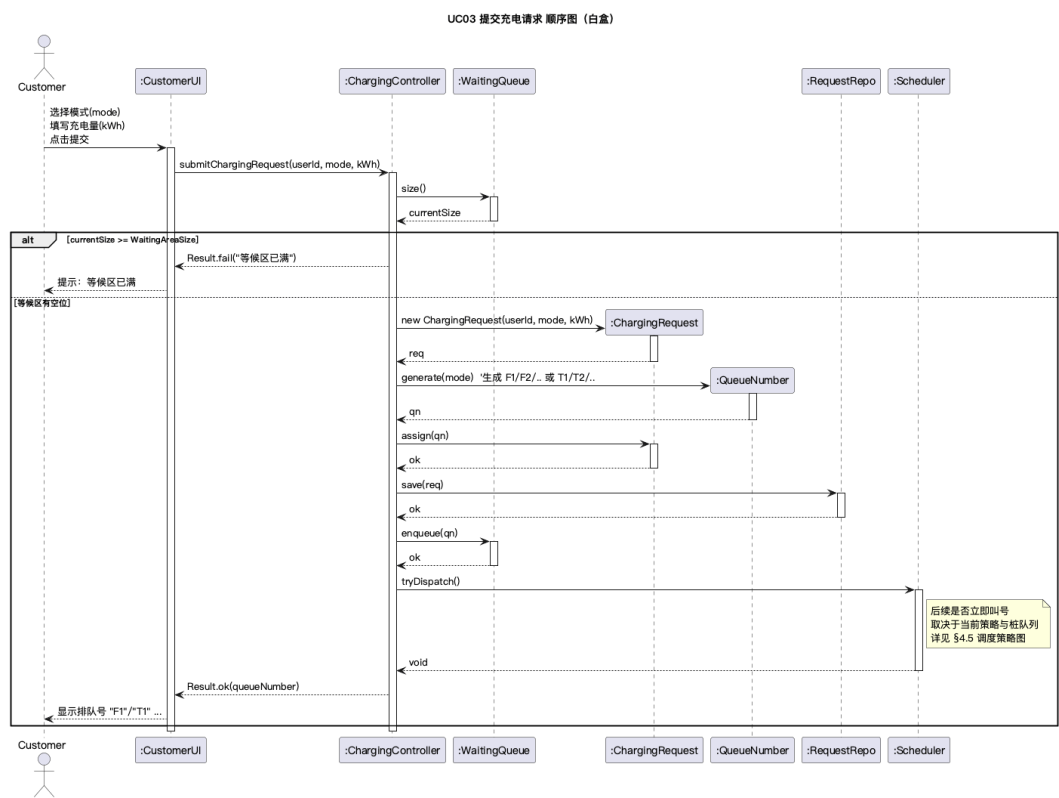


图 10 UC03 提交充电请求 顺序图

核心流程：

1. :ChargingController 先调 :WaitingQueue.size() 判断容量约束 (WaitingAreaSize) ；
2. 容量允许时创建 :ChargingRequest、生成 :QueueNumber (F/T 系列号)，落库 :RequestRepo；
3. 入队 :WaitingQueue 后调用 :Scheduler.tryDispatch() 触发后续叫号。

3.2.4 UC04 修改充电请求

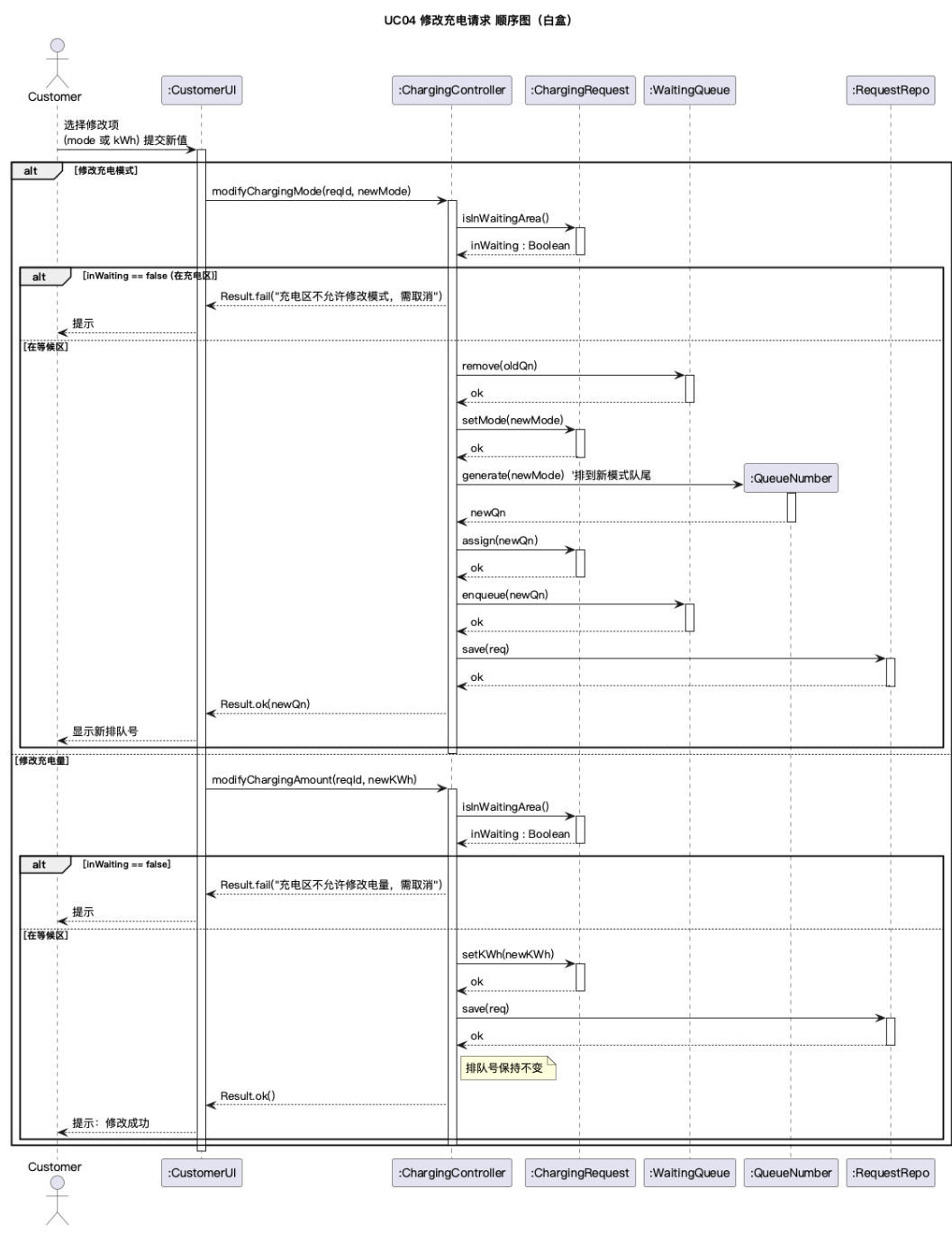


图 11 UC04 修改充电请求 顺序图

外层 alt 区分"改模式 / 改电量"; 内层 alt 用 :ChargingRequest.isInWaitingArea() 判断是否允许修改:

- 改模式: 在等候区允许, 需移出旧队列、生成新模式排队号、加入新模式队尾;
- 改电量: 在等候区允许且排队号保持不变;
- 充电区均不允许修改, 需取消重排 (流转到 UC05)。

3.2.5 UC05 取消充电

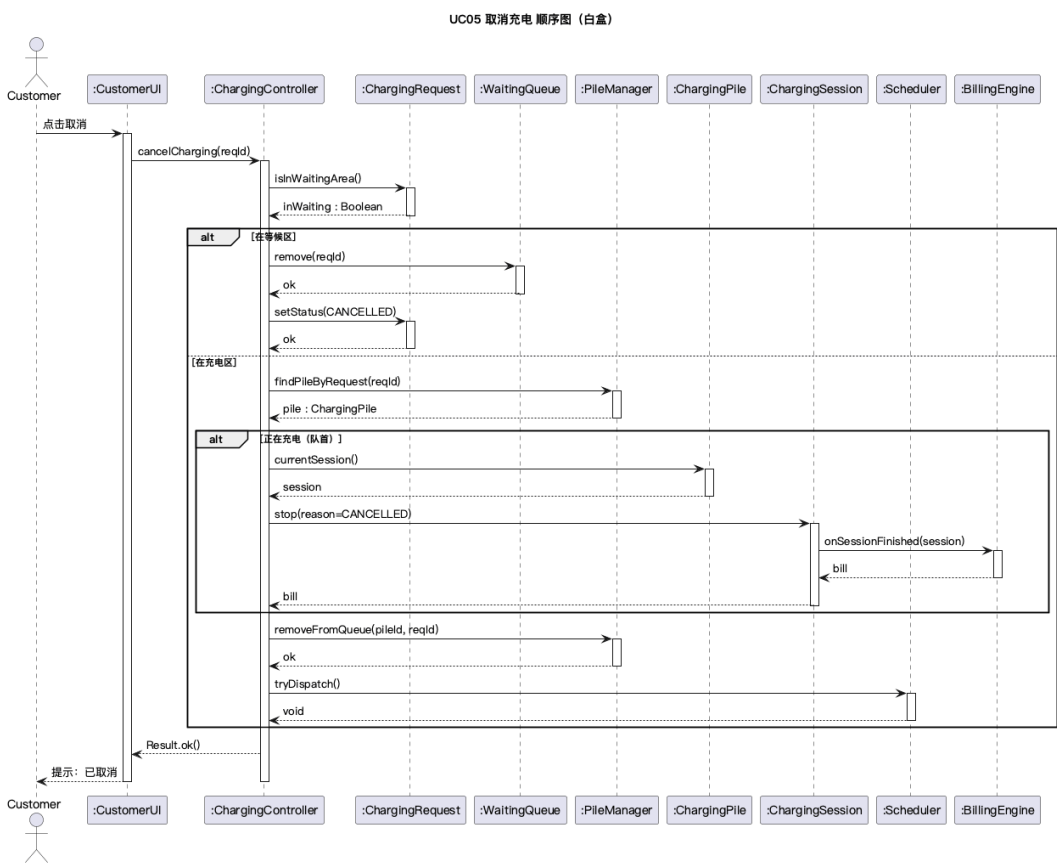


图 12 UC05 取消充电 顺序图

按等候区 / 充电区分别处理：在充电区时若正在充电（队首），先停止 :ChargingSession 并生成 :BillDetail（与 UC08 共享路径）；移除桩队列后调用 :Scheduler.tryDispatch() 重新调度。

3.2.6 UC06 查看本车排队号

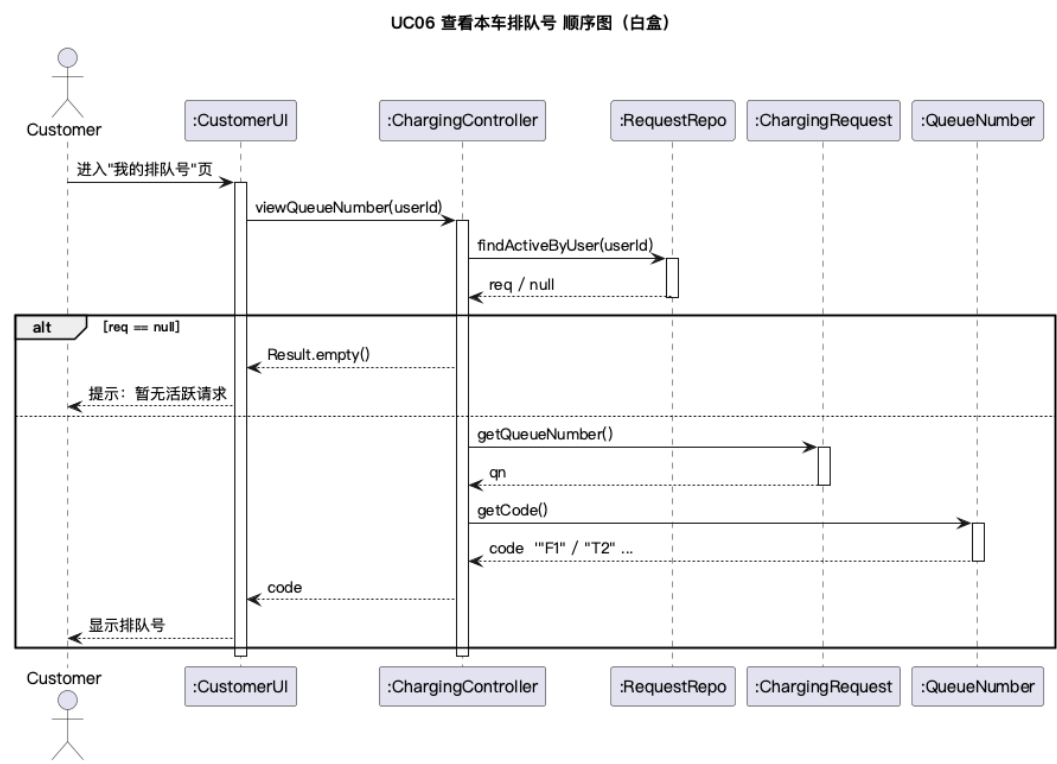


图 13 UC06 查看排队号 顺序图

只读查询；:RequestRepo.findActiveByUser() 找到当前请求，取 :QueueNumber.getCode() 返回。

3.2.7 UC07 查看本充电模式前车数

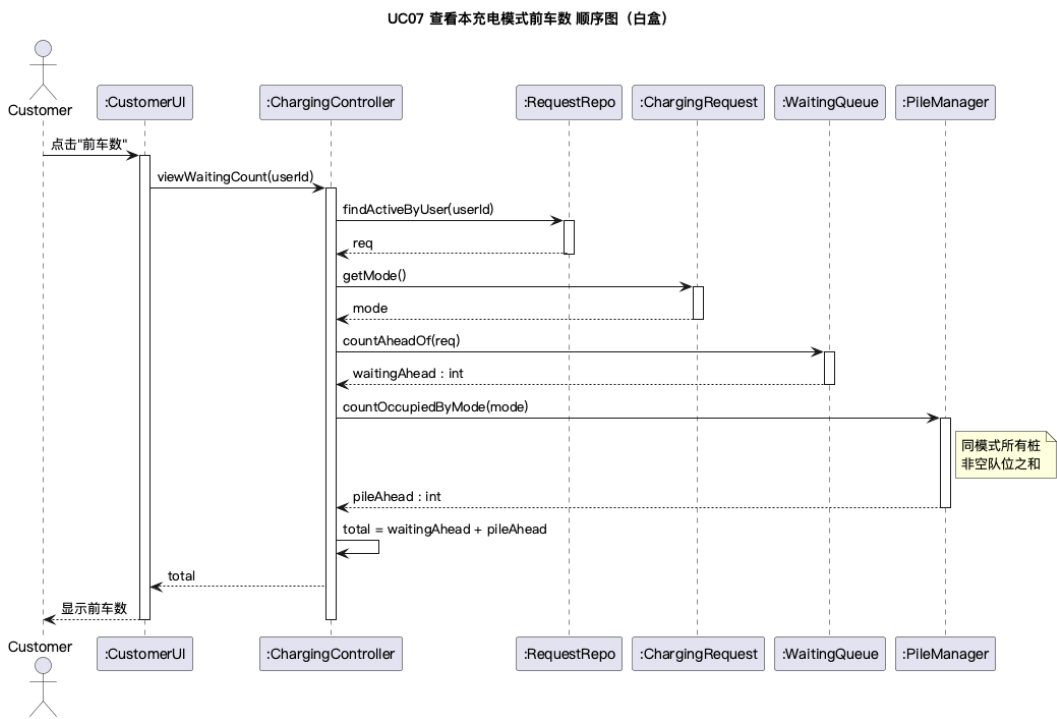


图 14 UC07 查看前车数 顺序图

前车数 =

`:WaitingQueue.countAheadOf(req) + :PileManager.countOccupiedByMode(mode);`

体现需求中"前车数"覆盖等候区+充电区同模式车辆。

3.2.8 UC08 结束充电

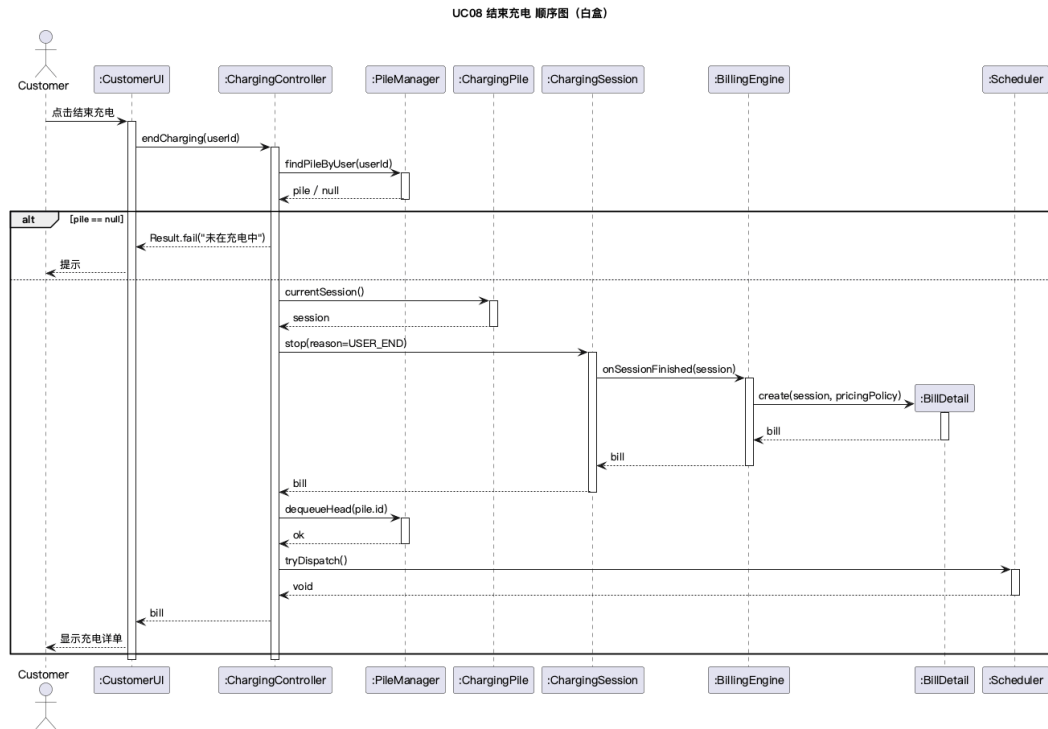


图 15 UC08 结束充电 顺序图

`:ChargingSession.stop(USER_END) → :BillingEngine.onSessionFinished()` 创建 `:BillDetail`; 随后从桩队列出队队首并触发 `:Scheduler.tryDispatch()`。

3.2.9 UC09 查看充电详单

UC09 查看充电详单 顺序图（白盒）

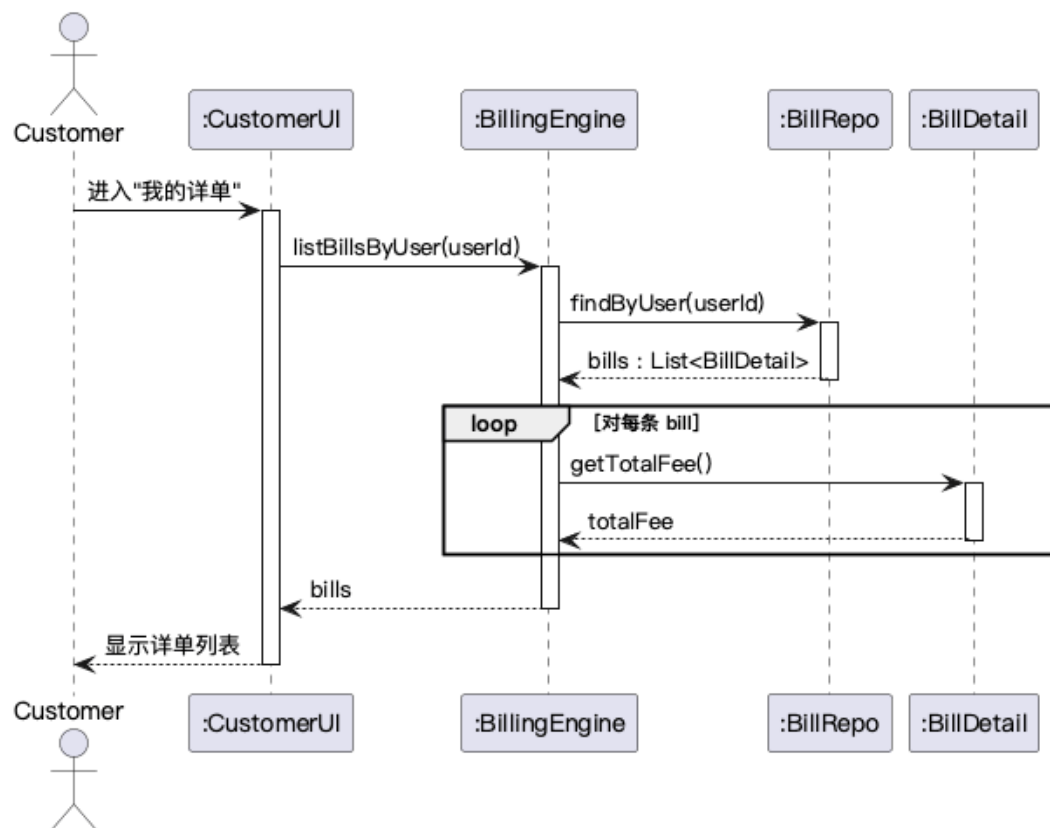


图 16 UC09 查看充电详单 顺序图

只读查询；loop 段遍历用户的所有详单并取总费用。

3.3 管理员域用例顺序图 (UC10~UC14)

3.3.1 UC10 启动/关闭充电桩

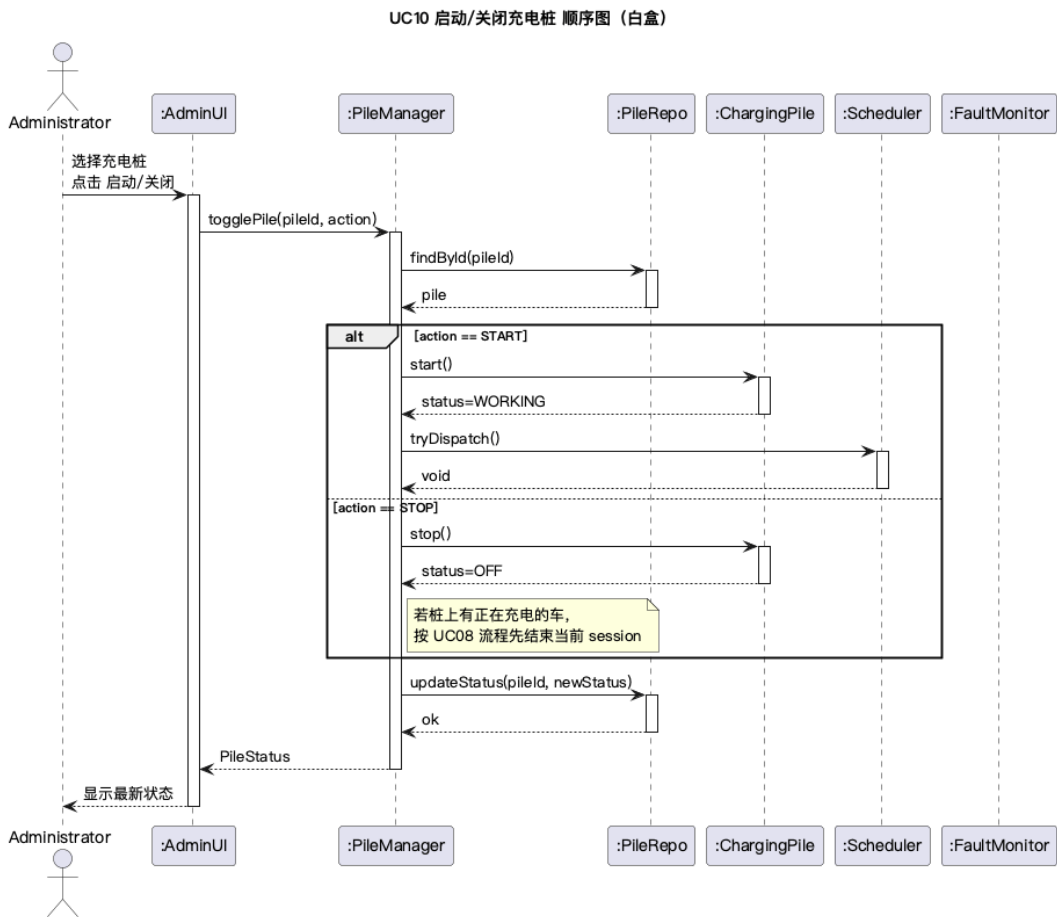


图 17 UC10 启动/关闭充电桩 顺序图

alt 区分 START/STOP; START 后立即 tryDispatch() 让新启动桩参与下一轮调度。

3.3.2 UC11 查看所有充电桩状态

UC11 查看所有充电桩状态 顺序图（白盒）

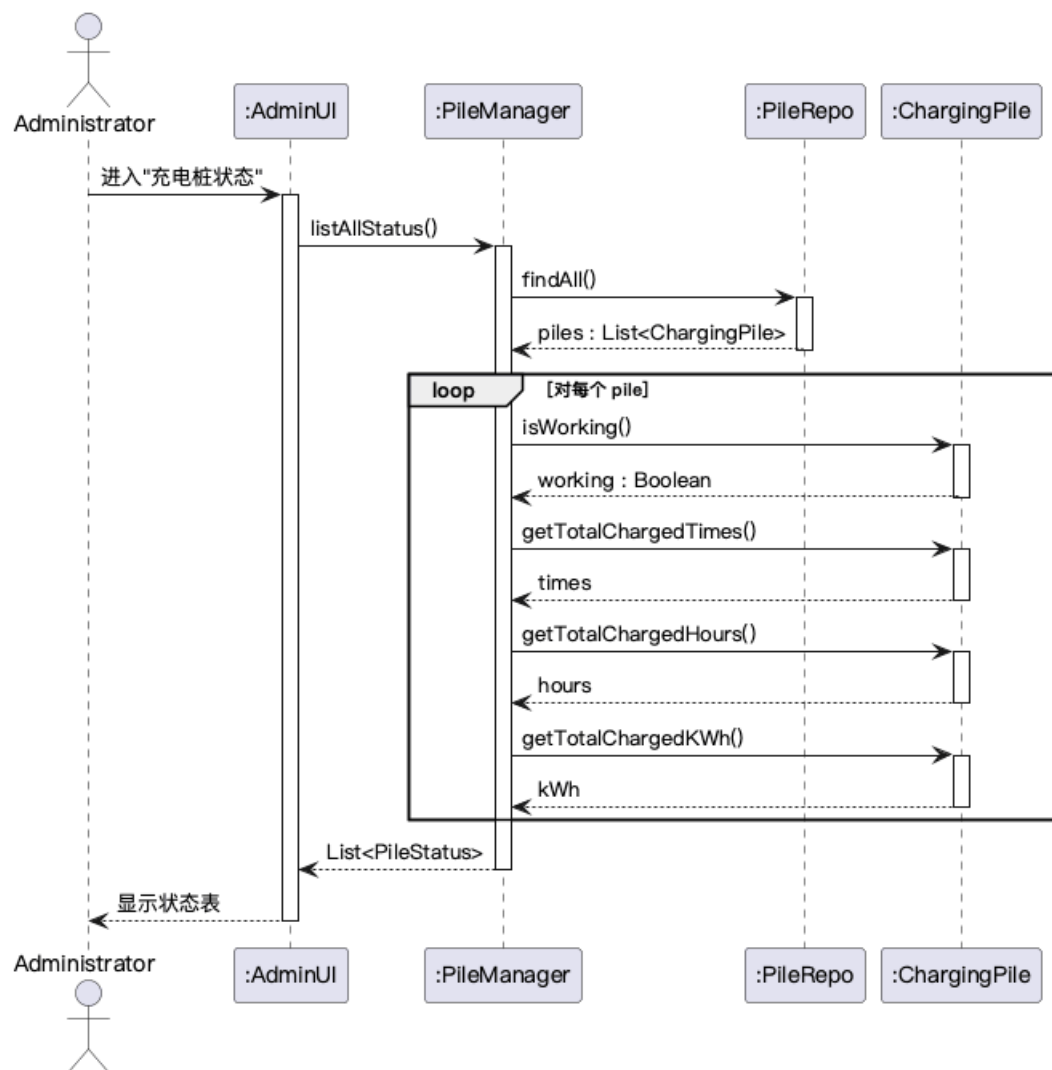


图 18 UC11 查看所有充电桩状态 顺序图

loop 段对每个 `:ChargingPile` 取 `isWorking()` + 三项累计统计，组装 `List<PileStatus>` 返回。

3.3.3 UC12 查看排队车辆信息

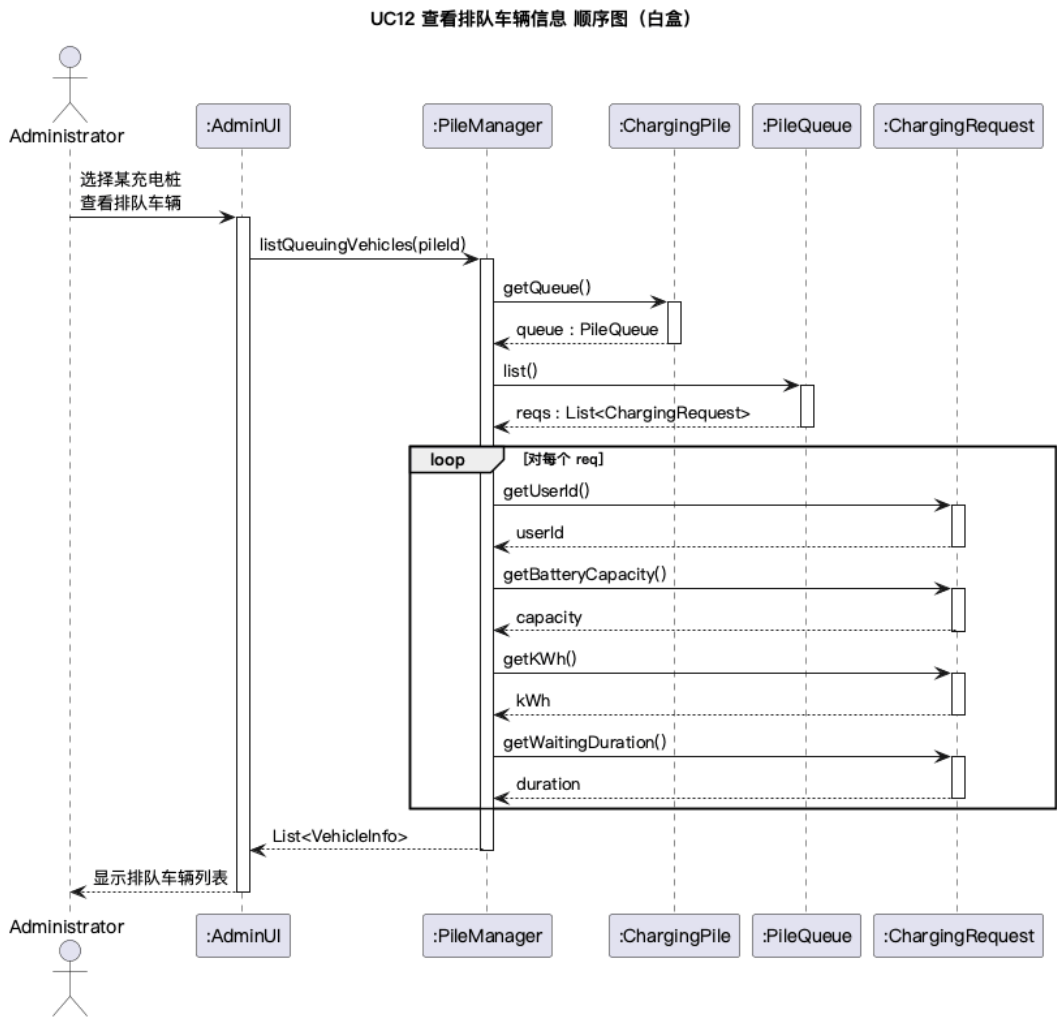


图 19 UC12 查看排队车辆信息 顺序图

定位指定 :PileQueue 后 loop 遍历队列内每个 :ChargingRequest，取 userId/电池容量/请求电量/排队时长，组装 List<VehicleInfo>。

3.3.4 UC13 查看运营报表

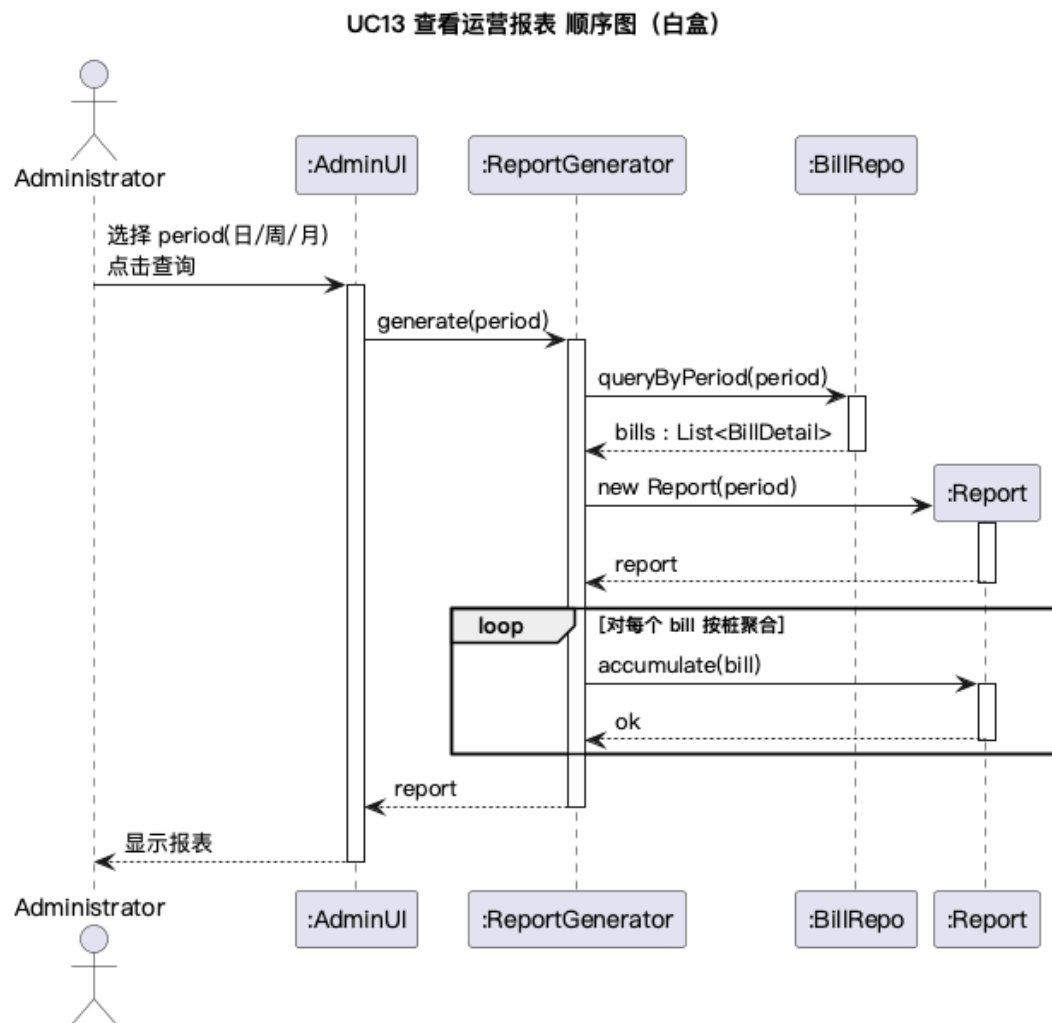


图 20 UC13 查看运营报表 顺序图

:ReportGenerator.generate(period) 从 :BillRepo 拉取该周期所有 :BillDetail, 按桩 accumulate() 聚合为 :Report。

3.3.5 UC14 切换调度策略

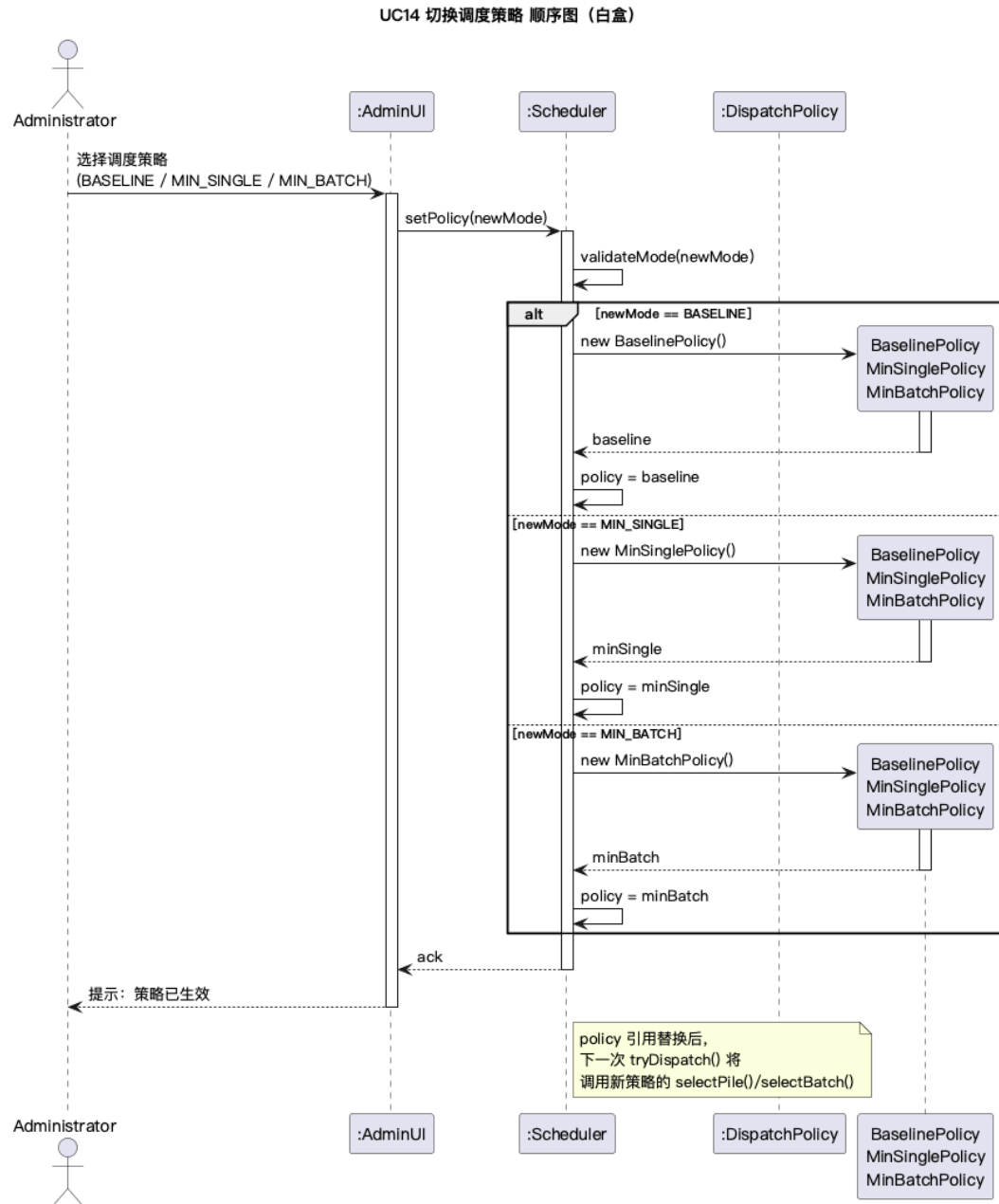


图 21 UC14 切换调度策略 顺序图

`:Scheduler.setPolicy(newMode)` 内部按 `alt` 三选一创建对应策略对象,把 `policy` 引用替换为新策略; 下一次 `tryDispatch()` 即生效 (Strategy 模式)。

3.4 充电桩故障调度顺序图

3.4.1 故障调度 — 优先级调度

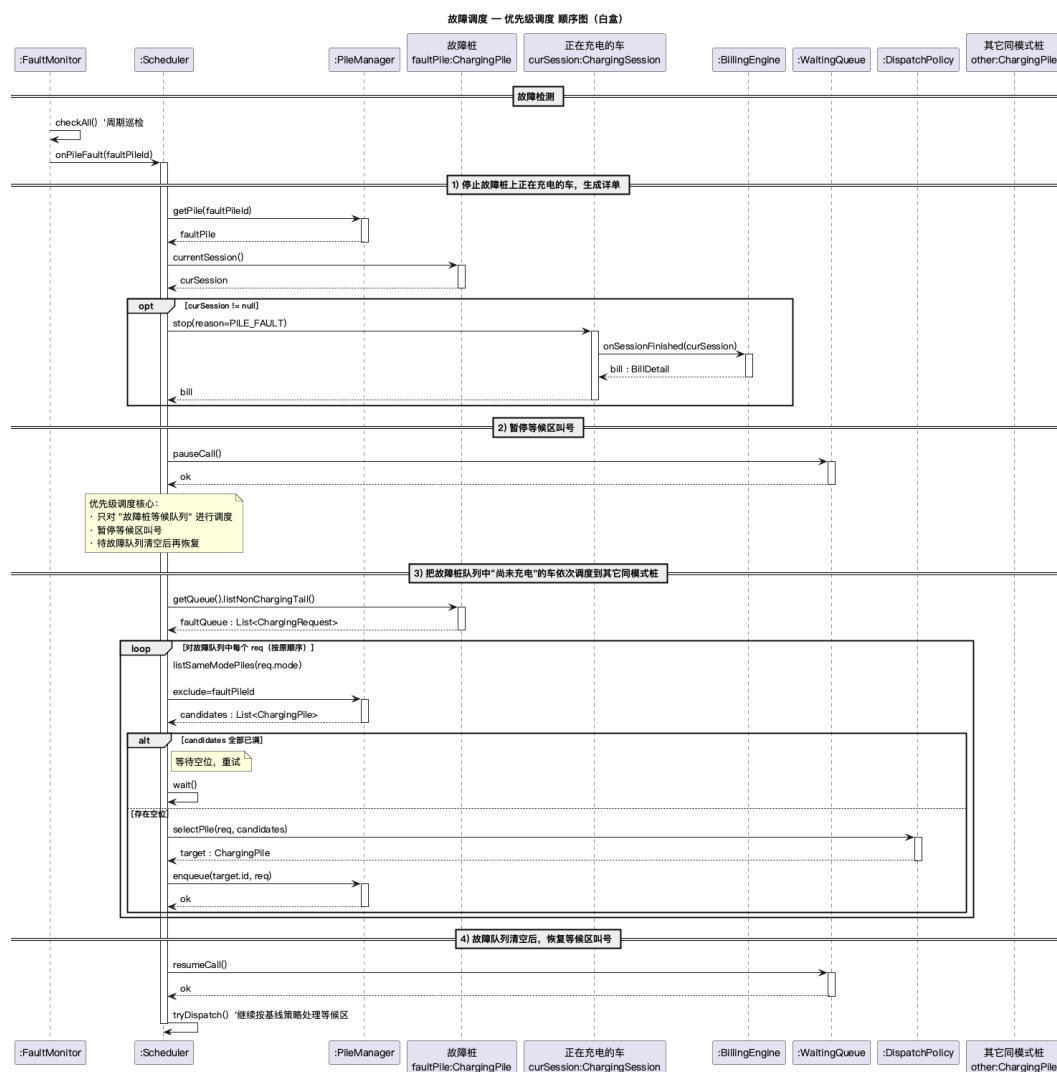


图 22 故障调度 - 优先级调度 顺序图

关键交互:

1. :FaultMonitor 周期 checkAll() 检测到 faultPileId , 通知 :Scheduler.onPileFault()。
2. 停止故障桩上正在充电的车: :ChargingSession.stop(PILE_FAULT) → :BillingEngine.onSessionFinished() 生成详单 (对应需求"本次充电过程对应一条详单")。
3. 暂停等候区叫号: :WaitingQueue.pauseCall(), 确保新车不在故障期间进入。
4. 优先调度故障队列: 从故障桩队列取出"尚未充电"的请求 (队首已停), 依次调用 :DispatchPolicy.selectPile() 把车移入其它同模式桩的空位; 若同模式桩全满则等待。
5. 故障队列清空后: :WaitingQueue.resumeCall() 恢复等候区叫号, 继续按基线策略运行。

3.4.2 故障调度 — 时间顺序调度

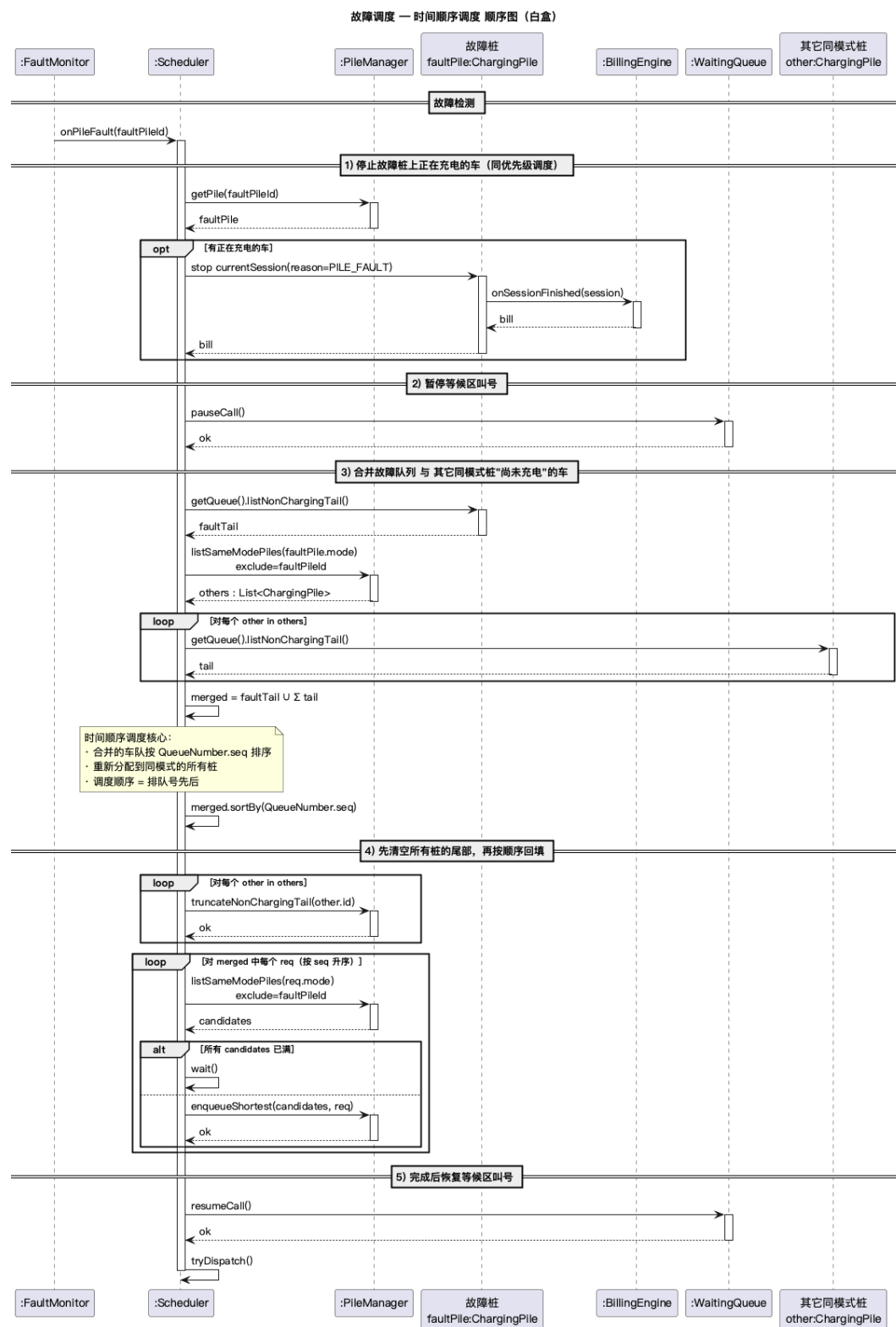


图 23 故障调度 - 时间顺序调度 顺序图

与优先级调度的区别:

- ⑩ 不仅取出故障队列的车辆, 还把其它同模式桩"尚未充电"的车辆一同卷入;

- ⑩ 合并队列后按 `QueueNumber.seq` 重新升序排序，再依次最短队列入队；
- ⑩ `truncateNonChargingTail()` 先清空各桩尾部，再按顺序回填，避免与原顺序冲突。
- ⑩ 调度过程亦在 `pauseCall()` / `resumeCall()` 之间执行。

3.4.3 故障恢复 — 重调度

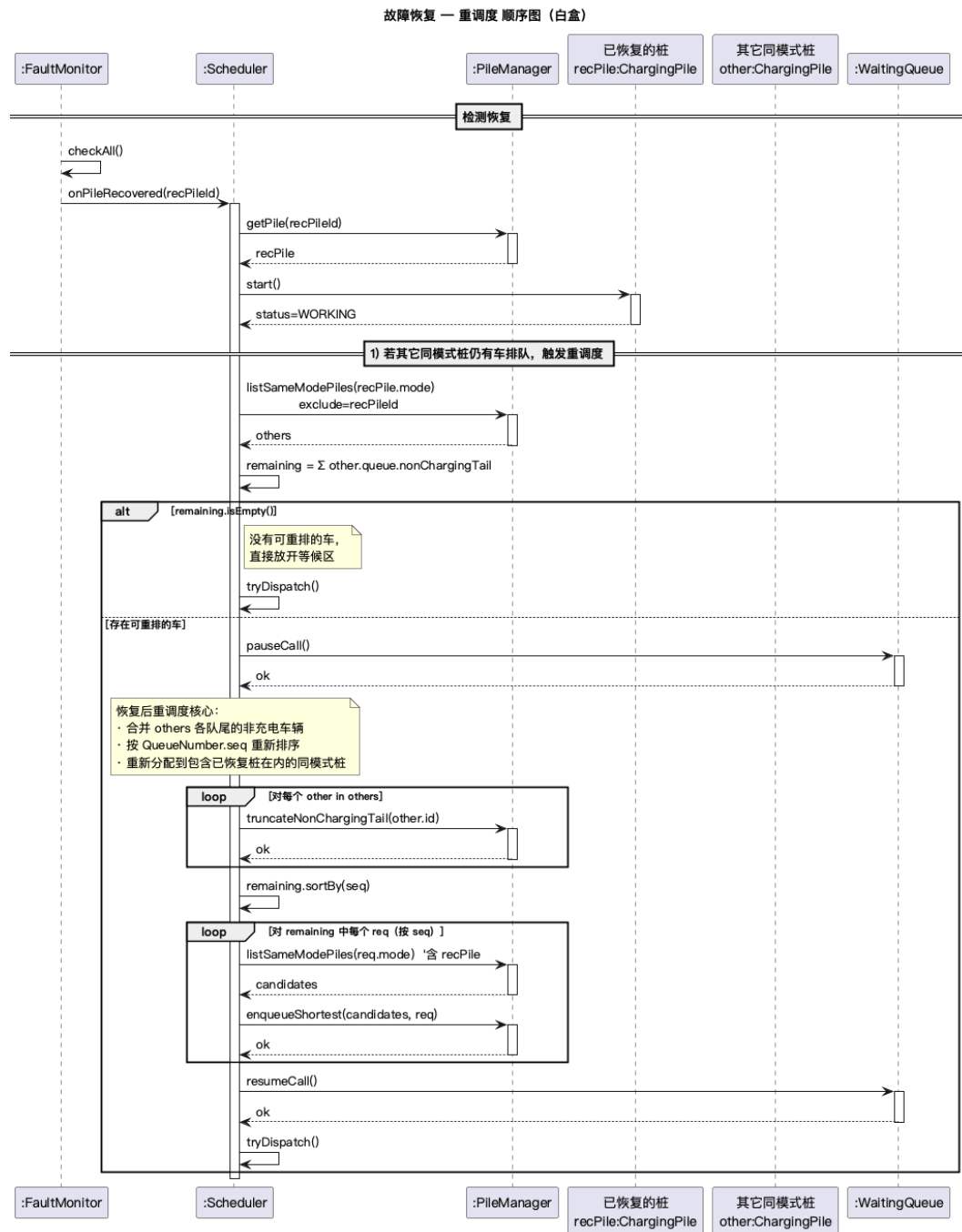


图 24 故障恢复重调度 顺序图

关键交互：

1. `:FaultMonitor.onPileRecovered()` 触发 `:Scheduler`;
2. `:ChargingPile.start()` 把恢复桩状态置 `WORKING`;
3. 检测是否需要重调度 (alt)：聚合其它同模式桩"尚未充电"的车，若为空则

直接 tryDispatch() 放开等候区；

4. 若存在剩余车辆：暂停等候区 → 合并并按 seq 排序 → 把所有同模式桩（含已恢复桩）作为候选 → 按最短队列重新分配 → 恢复等候区叫号。

需求中"若其它同类型充电桩中尚有车辆排队，则暂停等候区叫号服务"的条件以 alt remaining.isEmpty() 在图中显式呈现。

3.5 扩展调度策略顺序图

3.5.1 单次调度 — 总充电时长最短 (MinSinglePolicy)

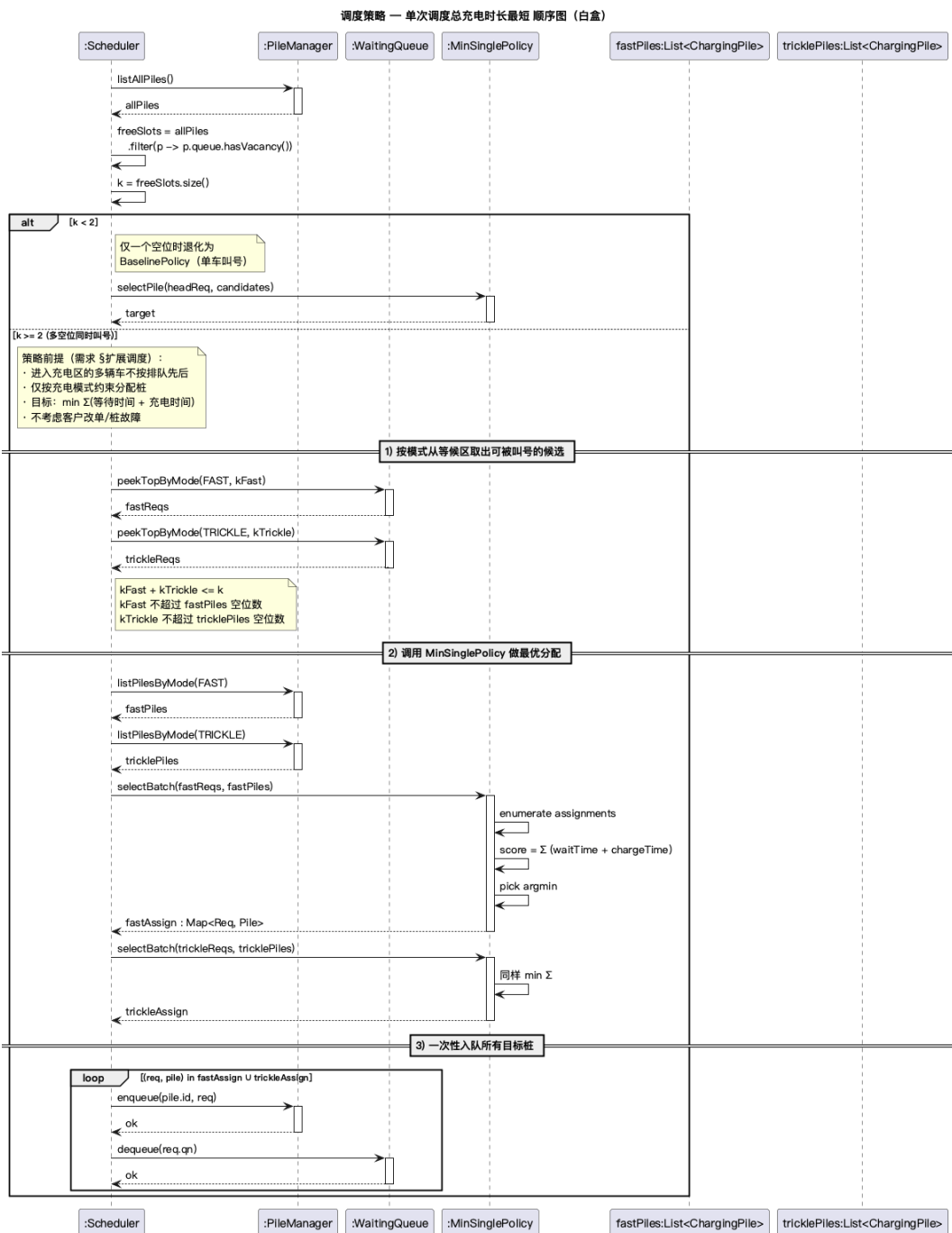


图 25 单次调度总充电时长最短 顺序图

需求条件在图中的呈现：

需求条件	图中表达
"充电区出现多个车辆空位时，可一次同时叫多个号"	<code>alt k >= 2; freeSlots = allPiles.filter(hasVacancy)</code>
"进入充电区的多辆车不考虑排队先后顺序"	不按 <code>QueueNumber.seq</code> 取，而是 <code>peekTopByMode(...)</code> 集合式取出
"按充电模式分配对应充电桩"	分两路 <code>selectBatch(fastReqs, fastPiles) / selectBatch(trickleReqs, tricklePiles)</code>
"满足完成充电总时长(Σ 等待+充电)最短"	<code>MinSinglePolicy</code> 内部 <code>score = \Sigma(waitTime + chargeTime)</code> 后 <code>pick argmin</code>

$k < 2$ 时退化为 `BaselinePolicy`（单车叫号）以保证策略一致性，是图中的容错分支。

3.5.2 批量调度 — 总充电时长最短 (MinBatchPolicy)

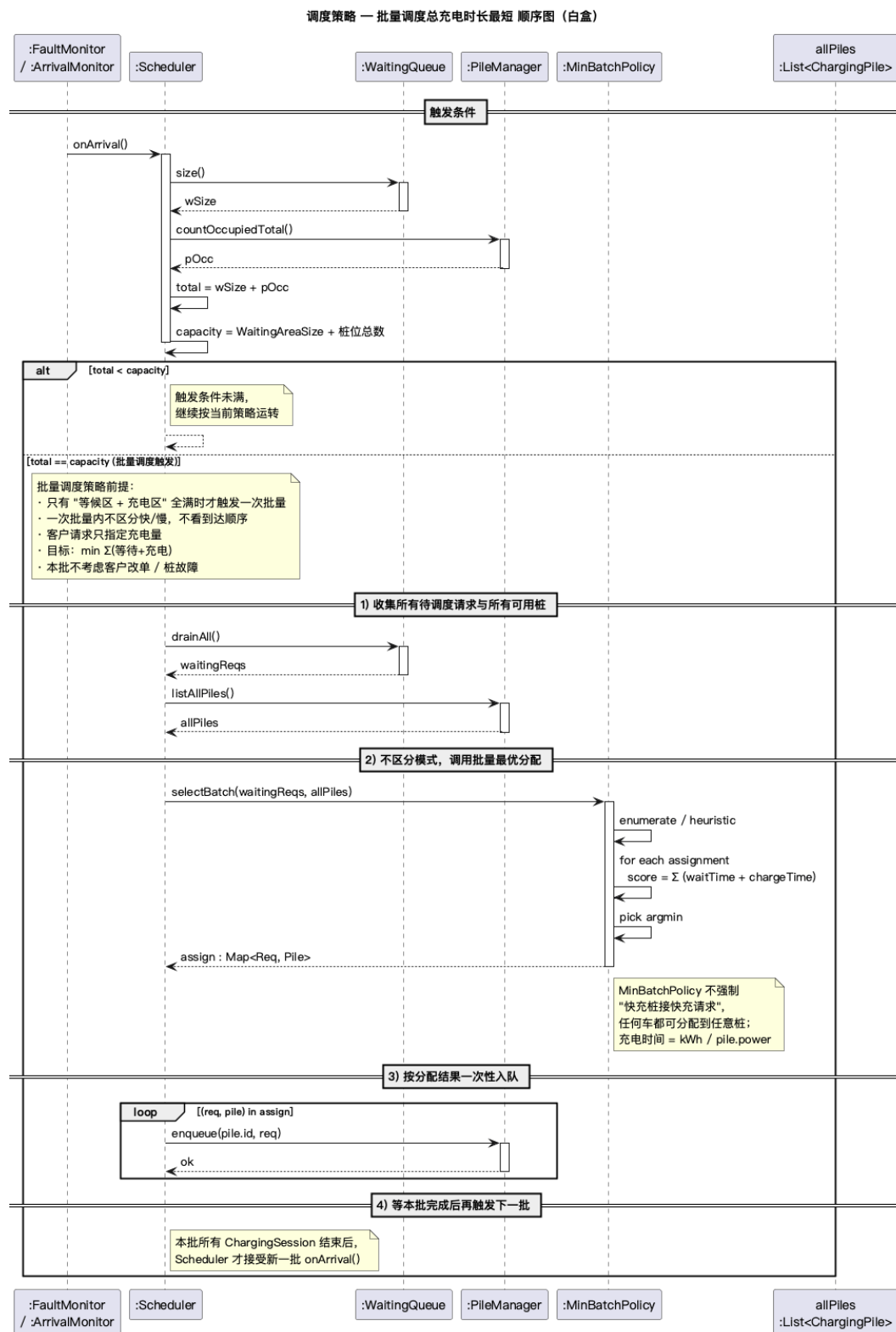


图 26 批量调度总充电时长最短 顺序图

需求条件在图中的呈现:

需求条件

图中表达